Computers & Security 172 (2027) 105167

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Full length article

GAN-based Tor obfuscated traffic identification on imbalanced datasets

Tianbo Lu, Pengfei Zhao, Yingjie Cai, Subatjan Abdulla *, Hanrui Chen

School of Computer Science (National Pilot Software Engineering School), Beijing University of Posts and Telecommunications, Beijing, China
Key Laboratory of Trustworthy Distributed Computing and Service (BUPT), Ministry of Education, Beijing, China

ARTICLE INFO

Keywords:
Tor
Obfuscated traffic
Traffic classification
Generative adversarial networks
Imbalanced datasets
Pluggable transports

ABSTRACT

Although deep learning models have demonstrated strong performance in identifying vanilla Tor traffic, the classification of obfuscated Tor traffic, particularly from newly emerged obfuscators, remains under-explored. A practical challenge is that real datasets are often limited and imbalanced, which can reduce deep learning performance on minority traffic classes. To bridge this gap, we adapt the GAN (Generative Adversarial Network) for a semi-supervised classification task of obfuscated Tor traffic. Leveraging the generator's capacity to synthesize minority-class feature representations, the discriminator is trained to classify traffic from popular obfuscation tools, including Obfs4, WebTunnel, Snowflake, Dnstt and Shadowsocks. We also curate a diverse dataset to evaluate our model against recent approaches, using Recall and F1-Score as key metrics. The reported results show competitive performance on near-balanced data; under controlled class imbalance, a backbone-controlled ablation reports higher point estimates for Tor-class recall and F1 when the generator is included.

1. Introduction

Tor is one of the largest deployed anonymity networks, comprising over 8000 volunteer-run relays and approximately 2,000,000 directly connecting users. It provides anonymity protection for users and hidden services using a multi-hop proxy, safeguarding their privacy. However, the same anonymity property also creates operational and security challenges for network measurement, policy enforcement, and abuse investigation, which has spurred extensive research into Tor traffic analysis.

As traffic analysis methods for the Tor network have matured and become widely deployed, vanilla Tor traffic, namely standard Tor traffic without any pluggable transport, can be identified by deep learning methods with high confidence. In response, contributors to the Tor projects have begun developing pluggable transports (PTs) that *encrypt*, *encapsulate*, *tunnel*, *mutate*, and *morph* traffic to circumvent censorship. This has led to an ongoing arms race between identification methods and obfuscation techniques, with each side continually evolving to outpace the other.

Previous work has shown that it is still possible to identify obfuscated Tor traffic through feature extraction and deep learning. However, these methods depend on high-quality datasets, which remain difficult to obtain for newly deployed PTs. To address this gap, we propose an obfuscated Tor traffic identification method that reduces the impact of limited and imbalanced training data. Our method extends the Generative Adversarial Network (GAN) (Goodfellow et al., 2020) to a semi-supervised classification task for obfuscated traffic, in which the generator's capacity for feature-space augmentation is leveraged to train the discriminator as a classifier for multiple Tor obfuscation tools. More specifically, there are three major issues with current datasets that motivate our work.

Data Shortage. There is currently a lack of widely recognized obfuscated Tor traffic datasets, which hinders the development and evaluation of corresponding identification models. To the best of our knowledge, recent research on obfuscated Tor traffic mainly relies on self-collected, small-scale datasets. In this context, GANs provide a feasible way to augment limited training data by learning the underlying feature distribution and generating synthetic feature representations.

Imbalanced Dataset. Class imbalance represents a critical challenge associated with classification tasks where the number of samples of each class is highly variable. This imbalance can lead to biased models that favor the majority class, resulting in poor generalization and reduced performance on minority classes.

Common methods for handling imbalanced datasets include undersampling and oversampling (L. Wang et al., 2021), but these techniques

* Corresponding author at: School of Computer Science (National Pilot Software Engineering School), Beijing University of Posts and Telecommunications, Beijing, China.
E-mail addresses: lutb@bupt.edu.cn (T. Lu), pengfeizhao316@bupt.edu.cn (P. Zhao), caiyj@bupt.edu.cn (Y. Cai), subat@bupt.edu.cn (S. Abdulla), chr@bupt.edu.cn (H. Chen).

https://doi.org/10.1016/j.cose.2026.105167
Received 27 April 2026; Received in revised form 18 August 2026; Accepted 11 September 2026
Available online 17 September 2026
0167-4048/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

can either discard valuable data or introduce a potential risk of overfitting. A more targeted approach involves using GANs to generate synthetic representations for minority classes, helping balance the training signal while preserving the original feature distribution. This approach addresses the imbalance issue by diversifying the minority-class information available during training.

Traffic Noise. Although most traffic is captured in a controlled environment where only a single application runs at a time, captured traffic can still include unrelated packets or variations caused by network dynamics. Such effects may skew the underlying distribution necessary for accurate classification. In this work, we minimize background traffic during collection and focus our experimental evaluation on data scarcity and class imbalance, while leaving controlled noise-injection analysis as an important extension.

In addressing the identified gaps, our main contributions include:

- We deploy Tor clients as proxies with various obfuscation PTs, including Obfs4, WebTunnel, Snowflake, Dnstt and Shadowsocks. We then use automation scripts to generate and collect a multi-PT dataset of obfuscated traffic.
- We propose a semi-supervised GAN for class-imbalanced obfuscated traffic classification, where the generator supplies synthesized minority-class feature representations during training and the discriminator is trained to distinguish between different types of obfuscated Tor traffic, vanilla Tor traffic and normal encrypted traffic.
- We evaluate the method on a self-collected dataset through end-to-end comparisons and a backbone-controlled ablation. The ablation reports higher point estimates for minority-class recall and F1 when the generator is included.
- We analyze important implementation choices, including batch normalization, layer normalization, and trace segmentation, and discuss the ethical and practical scope of obfuscated Tor traffic identification.

The remainder of this paper is structured as follows. Section 2 provides essential background knowledge about popular obfuscation tools in Tor, forming the foundation for our proposed obfuscated traffic identification method. Section 3 reviews prior research on obfuscated traffic detection and the application of GANs in related fields. Section 4 elaborates on the architecture of the generator and discriminator in our proposed GAN-based method. Section 5 presents the experimental setup and evaluation metrics. Section 6 reports the experimental results. Section 7 discusses ethical considerations and the scope of the evaluation. Section 8 concludes the paper.

2. Preliminaries

Currently, a variety of obfuscation tools employing different techniques are available for Tor, including Obfs4, WebTunnel, Snowflake, Dnstt, and Shadowsocks. In this study, we focus on distinguishing obfuscated Tor traffic generated by these tools from vanilla Tor traffic and normal encrypted traffic. Here, vanilla Tor traffic refers to standard Tor traffic that does not use a pluggable transport; it is different from the Tor reachability tests sometimes used to diagnose whether direct Tor connections are blocked. Snowflake uses WebRTC with DTLS, Dnstt encapsulates traffic in DNS messages, and WebTunnel and Shadowsocks attempt to blend Tor traffic into widely deployed application protocols. These differences make multi-class obfuscated traffic identification more challenging than detecting vanilla Tor traffic alone.

2.1. Obfs4

Obfs4 introduces random, non-deterministic elements to each connection to effectively conceal the underlying structure of the traffic, thus preventing a passive adversary from telling what protocol is in use based on message contents. This randomized approach disrupts deep packet inspection (DPI) and various forms of signature-based traffic analysis.

Obfs4 relies on the use of volunteer-operated bridge relays that are deliberately kept off public listings. The bridge relays also incorporate robust anti-probing defenses to prevent active detection. Specifically, an adversary should not be able to verify the existence of an Obfs4 bridge without obtaining its Node ID and public key.

When Obfs4 is enabled, Tor establishes a connection to bridge relays through an Obfs4 client. Following a handshake procedure in which encryption keys are exchanged, the Obfs4 client appends a random size of padding data to obscure packet size and encapsulates as payload the Tor cells within the Obfs4 layer before sending these sealed packets to the bridge. Upon receiving the packets, the bridge removes the Obfs4 layer and forwards the cells to the next relay in the Tor circuit in lieu of the guard relay. The rationale behind this configuration is to ensure that the total number of relays within any given Tor circuit remains three for uncompromised network efficiency.

2.2. WebTunnel

As previously mentioned, Obfs4 strives to scramble the traffic patterns with the aim of transforming them into random, unrecognizable data streams. However, HTTPT (Frolov and Wustrow, 2020) posits that not resembling any popular protocol can itself be seen as anomalous behavior, potentially leading to the detection and consequent blocking of bridges. A more prudent strategy is to blend in with ubiquitously deployed protocols such as HTTPS.

Inspired by HTTPT, WebTunnel is implemented as a Tor obfuscator to mimic encrypted web traffic (HTTPS). It operates by encapsulating the payload connection within a WebSocket-like HTTPS connection, making it appear to network observers as a benign HTTPS (WebSocket) connection.

To effectively defend certain active probing techniques, a WebTunnel bridge can coexist with a website on the same network endpoint, sharing the same domain, IP address, and port. This coexistence allows a standard reverse proxy to forward both ordinary web traffic and WebTunnel. Access to the WebTunnel service is granted only to those who can prove knowledge of a secret typically obtained out of band.

2.3. Snowflake

Snowflake (Bocovich et al., 2024) enables anonymous access to the Tor network by routing user traffic through lightweight proxies operated by volunteers. These proxies relay traffic to Tor bridges that are otherwise inaccessible due to censorship. Clients and proxies exchange essential information and establish point-to-point connections via a broker server; once the connection is established, further communication occurs independently of the broker. Snowflake's effectiveness in censorship circumvention stems from its lightweight, highly dynamic, and large-scale proxy pool. Proxies are easy to deploy. For example, JavaScript code can be embedded in participating websites, allowing users visiting these sites to temporarily serve as Snowflake proxies.

To maintain session stability despite proxy volatility, Snowflake employs a session-based mechanism. When a proxy becomes unavailable, the client seamlessly switches to another, ensuring continuity for upper-layer applications. However, this process results in multiple UDP flows at the network level, complicating traffic identification and making it difficult to distinguish Snowflake traffic from background WebRTC traffic.

2.4. Dnstt

Dnstt encapsulates Tor traffic within DNS queries and responses to evade detection based on conventional Tor traffic signatures. The client splits Tor data into small chunks, encodes them, and transmits them via DNS queries to a recursive resolver. Since the resolver cannot process these queries directly, it forwards them to an authoritative name server running the Dnstt server. The Dnstt server decodes the DNS queries, extracts the embedded Tor payload, and relays it through the Tor network. Response data is similarly embedded within DNS response fields and returned to the client via standard DNS resolution paths, enabling covert communication in censored environments.

Because DNS traffic is difficult to block completely, Dnstt effectively circumvents traffic-based censorship while leveraging the resilience of existing DNS infrastructure. However, due to inherent size limitations of DNS queries, Dnstt often suffers from significant performance constraints.

2.5. Shadowsocks

Originally designed to bypass internet censorship, Shadowsocks is an encrypted proxy protocol that has also been adopted to obfuscate Tor traffic. It encrypts Tor data to prevent identification during transmission. The client first encrypts the traffic using either a pre-shared key or key exchange mechanisms to ensure data confidentiality and integrity. The encrypted traffic is then transmitted via standard TCP or UDP to a Shadowsocks server, which decrypts and forwards the data to censored Tor bridges, enabling anonymous access.

Shadowsocks' resistance to censorship relies on traffic obfuscation and dynamic port mechanisms. Some implementations (e.g., V2Ray) can further disguise the traffic to resemble HTTPS or other legitimate application protocols, reducing the risk of detection. Additionally, features such as dynamic port allocation and traffic padding make it difficult for deep packet inspection (DPI) systems to extract stable signatures.

3. Related work

Obfuscated Traffic Identification: The identification of Obfs4, Meek and FTE has been thoroughly examined in previous work. Meek is a domain-fronting-based pluggable transport that makes Tor connections resemble ordinary HTTPS traffic to major web services. J. Wang et al. (2021) used random forest to rank and select the most informative flow features, based on which C4.5, RandomForest and KNN algorithms were applied to identify Meek traffic. Xu and Zou (2021) employed a sliding window technique to extract 12 time-based and packet size-based features from the network stream and applied several machine learning algorithms to effectively distinguish Obfs4, Meek, and FTE obfuscated traffic. Yao et al. (2020) proposed a classification model based on Gaussian mixture models and hidden Markov models, achieving high accuracy across a variety of obfuscated PTs.

Since the emergence of HTTPT and WebTunnel is relatively recent, research on identifying them is scarce. Xue et al. (2024) proposed a protocol-agnostic approach to proxy detection utilizing the shared characteristics of encapsulated TLS handshakes. They tested their method against 23 obfuscated proxy configurations, including HTTPT, and achieved true positive rates all exceeding 70%.

Qu et al. (2023) designed an input-agnostic hierarchical deep learning framework for traffic fingerprinting, which is capable of handling heterogeneous traffic with minimal traffic customization (e.g. padding, tailoring, feature extraction) while hierarchically abstracting relevant features. The framework was evaluated on five datasets across five different traffic fingerprinting tasks, including website fingerprinting on Shadowsocks-obfuscated traffic, and achieved results comparable to or even better than state-of-the-art methods.

Al-E'mari et al. (2023) proposed a GAN-based approach for detecting obfuscated Tor traffic by combining bidirectional generative adversarial learning with a vision-transform-style representation. Their work demonstrates the usefulness of adversarial representation learning for obfuscated Tor detection. In contrast, our work focuses on PT-level multi-class identification under class imbalance. Rather than primarily converting traffic into visual representations, our discriminator models the packet-flow-trace hierarchy with CNN and recurrent modules, while the generator performs class-conditioned feature-space augmentation for minority classes.

GAN-based Traffic Obfuscation: Several studies focus on reinforcement of obfuscation proxy technologies using adversarial approaches. Sheffey and Aderholdt (2019) trained a GAN network to minimally modify statistical signatures of Tor traffic in a way that hinders classification. Zou et al. (2023) employed Temporal Convolutional Networks (TCNs) to construct a GAN that learns traffic features to achieve effective mimicry of target traffic. Although there is a wealth of studies on PT improvement, research on GAN-based obfuscated traffic identification is limited.

GAN-based Semi-supervised Learning: Ever since the inception of Generative Adversarial Networks (GANs) (Goodfellow et al., 2020), there has been a continuous effort to improve their performance and architecture. Specifically, a number of papers focus on modifying GANs for unsupervised or semi-supervised learning. Springenberg (2015) proposed catGAN to learn a discriminative classifier from unlabeled or partially labeled data. Radford et al. (2015) introduced DCGAN for unsupervised tasks, employing Convolutional Neural Networks (CNNs) for both the generator and discriminator. Salimans et al. (2016) developed an effective semi-supervised learning model based on some of these techniques.

4. Model

4.1. Inputs and outputs

In our proposed model, a *trace* serves as the input sample for the discriminator, while the generator produces trace-level feature representations aligned with the discriminator's feature space. Hierarchically, a *trace* comprises several related *flows*, each consisting of a sequence of *packets*.

Packet: A packet is the fundamental unit of data transmission on a network.

To prepare the traffic data for the discriminator network, the process of *packet vectorization* serves as the initial processing step for the raw traffic. During the preprocessing phase, we extract metadata (e.g., packet size, arrival time) from each packet to construct a packet vector (PV), which is formally defined as follows:

$$PV_{i,j} =< i, j, s_{i,j}, (t_{i,j} - t_0), (t_{i,j} - t_{i,j-1}) > \tag{1}$$

where j and i denote the packet's ID and the flow ID it belongs to, respectively; $s_{i,j}$ represents packet size with sign indicating its direction (i.e., incoming or outgoing); $(t_{i,j} - t_0)$ is the time interval between packets $p_{i,j}$ and the very first packet in the trace; and $(t_{i,j} - t_{i,j-1})$ is the time interval between packets $p_{i,j}$ and $p_{i,j-1}$. For the first packet in each flow, we define $t_{i,0} = t_{i,1}$.

In practice, i and j are not necessarily required given that the Long Short-Term Memory (LSTM) we employed preserves the relative order of elements in the input sequence. For the sake of conciseness, $PV_{i,j}$ is therefore rewritten as:

$$PV_{i,j} =< s_{i,j}, (t_{i,j} - t_0), (t_{i,j} - t_{i,j-1}) > \tag{2}$$

Flow: A flow is typically defined as a sequence of packets sharing the same protocol, source/destination IP addresses and source/destination ports.

However, the Tor client multiplexes multiple TCP streams over a single circuit and multiple circuits over a single TLS connection as an

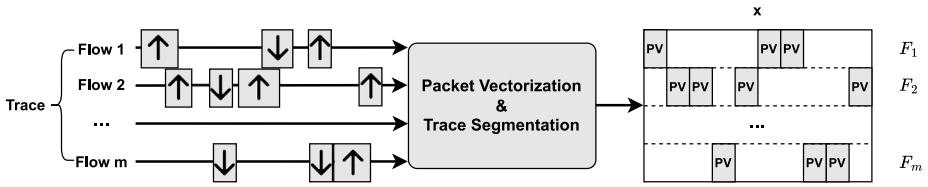


Fig. 1. An example of the hierarchical structure of a trace.

optimization as well as a countermeasure to traffic fingerprinting. With circuit multiplexing, communication data within the same connection window are likely to share the same connection, making it difficult to clearly demarcate the boundaries between flows.

In this case, a time-based trace segmentation method is used to partition flows and traces. A detailed discussion of trace segmentation is provided in Section 5.2. After the aforementioned packet vectorization process, a flow of length m can be represented as a sequence of packet vectors:

$$F_i = (PV_{i,1}, PV_{i,2}, \dots, PV_{i,m}) \tag{3}$$

Trace: A trace is a combination of multiple flows associated with a single obfuscator type. These traces serve as input samples for the discriminator. This trace-level view is consistent with recent traffic-analysis research that treats traces, rather than isolated packets or single flows, as the analysis unit for preserving temporal and cross-flow context (Qu et al., 2026). A trace or a sample consisting of n flows is formulated as:

$$\boldsymbol{x} = (F_1, F_2, \dots, F_n) \tag{4}$$

A schematic example is presented in Fig. 1. The left-hand side illustrates the raw traffic, where the width of the rectangles corresponds to packet sizes, and the upward and downward arrows represent packet directions. In contrast, the right-hand side depicts the hierarchical structure of a trace after *packet vectorization* and *trace segmentation*.

4.2. Overall design

The original GAN (Goodfellow et al., 2020) solves a two-player zero-sum minimax game between a generator model G and a discriminator model D defined by the value function $V(G, D)$, where $D(\boldsymbol{x})$ represents the probability that $\boldsymbol{x}$ is real rather than synthesized by $G(\boldsymbol{z})$, and $\boldsymbol{z}$ is noise sampled from the distribution $p_z(\boldsymbol{z})$.

$$\begin{aligned}\min_G \max_D V(G, D) = \; & \mathbb{E}_{\boldsymbol{x}\sim p_{data}(\boldsymbol{x})}[\log D(\boldsymbol{x})] \\ & + \mathbb{E}_{\boldsymbol{z}\sim p_z(\boldsymbol{z})}[\log(1 - D(G(\boldsymbol{z})))]\end{aligned} \tag{5}$$

To mitigate the vanishing-gradient problem for G, the authors of GAN highlighted the alternative minimization of $\mathcal{L}_G$ for the generator.

$$\mathcal{L}_G = -\mathbb{E}_{\boldsymbol{z}\sim p_z(\boldsymbol{z})}[\log D(G(\boldsymbol{z}))] \tag{6}$$

As suggested by Salimans et al. (2016), the formulations can be readily adapted to a semi-supervised multi-class classification setting. Suppose that a sample $\boldsymbol{x}$ can be categorized into K classes; we can extend this set of classes by introducing an additional "generated" class $y = K+1$. Instead of having the discriminator predict the probability of a sample $\boldsymbol{x}$ being real or synthesized, we modify it to produce a $(K+1)$-dimensional vector $\hat{\boldsymbol{y}}$, where each component represents the probability that $\boldsymbol{x}$ belongs to a specific class.

Assuming that the real trace-derived representations and synthesized feature representations are fed to the discriminator together, the discriminator can be trained by incorporating a cross-entropy term between the ground truth distribution $\boldsymbol{y}$ and the predicted conditional distribution $p(y \mid \boldsymbol{x}, D)$. Specifically, the cross-entropy for a labeled data pair $(\boldsymbol{x}, \boldsymbol{y})$ is given as

$$CE\big[\boldsymbol{y}, p(y \mid \boldsymbol{x}, D)\big] = -\sum_{i=1}^{K} y_i \log p(y = i \mid \boldsymbol{x}, D) \tag{7}$$

where the label $\boldsymbol{y}$ is one-hot encoded. The resulting loss function for training the discriminator is:

$$\begin{aligned}\mathcal{L}_D = & \mathbb{E}_{\boldsymbol{x},\boldsymbol{y}\sim p_{\text{data}}(\boldsymbol{x},\boldsymbol{y})}\Big[CE[\boldsymbol{y}, p(y \mid \boldsymbol{x}, D)]\Big] \\ & - \mathbb{E}_{\boldsymbol{x}\sim G}\Big[\log p(y = K+1 | \boldsymbol{x}, D)\Big]\end{aligned} \tag{8}$$

By minimizing the loss function in Eq. (8), the discriminator aims to achieve the following objectives:

Objective 1: Accurately predict the class labels of real samples.

Objective 2: Confidently identify samples synthesized by the generator.

There are additional issues to consider in the design of the generator. Prior research (Arjovsky and Bottou, 2017) highlighted two key issues in the original GAN design: instability and mode collapse. Instability refers to the difficulty of reaching a stable adversarial training state between the discriminator and the generator. This can be mitigated by introducing batch normalization and layer normalization into networks, as evaluated in Section 6.4. Mode collapse happens when the generator tends to produce a limited set of samples that can confidently deceive the discriminator in order to optimize its loss function, resulting in reduced diversity of the synthesized data. Mode collapse is particularly detrimental in our case because it limits the generator's ability to represent minority-class feature patterns and consequently compromises the original design objectives.

ACGAN (Odena et al., 2017) employed class-conditioned generation to enhance the diversity of synthesized samples, thus mitigating mode collapse. By embedding one-hot encoded class labels into the noise vector, the generator is guided to produce data that aligns with the specified class distribution. We take a similar technique in designing the generator: it is rewarded only when the synthesized samples successfully deceive the discriminator into assigning the expected labels. Consequently, the generator's loss function is formulated as follows:

$$\mathcal{L}_G = -\mathbb{E}_{\boldsymbol{x}\sim G(c,\boldsymbol{z})}\Big[\log p(y = c | \boldsymbol{x}, D)\Big] \tag{9}$$

As a result, the overall architecture of our GAN is given in Fig. 2. We will detail the neural network structures of the discriminator and generator in the following sections.

4.3. Discriminator

The discriminator adopts a structure akin to the hierarchical deep learning framework proposed by Qu et al. (2023) to capture traffic structure. As depicted in Fig. 3, the discriminator processes a trace of packet vectors (PVs) as input. The trace is sequentially passed through a convolutional neural network (CNN) and two Long Short-Term Memory (LSTM) networks to extract features at the packet, flow, and trace levels, respectively. Ultimately, the feature vector of the input trace is fed into a fully connected (FC) neural network, which outputs a vector of class prediction probabilities.

Packet vector compression through convolutional neural network. Due to the challenges posed by the vanishing/exploding gradient problem in LSTM networks and the computational intractability of processing very long sequences of packet vectors, we incorporate a 1D CNN to extract high-dimensional features from the original sequence, consequently compressing the input trace. The resulting shorter sequences of vectors are referred to as *compressed packet vectors* (CPV).

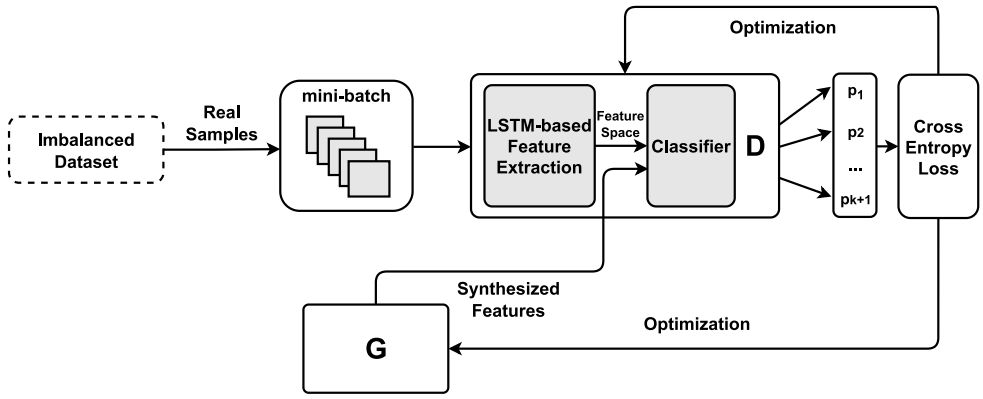


Fig. 2. The overall architecture of our model.

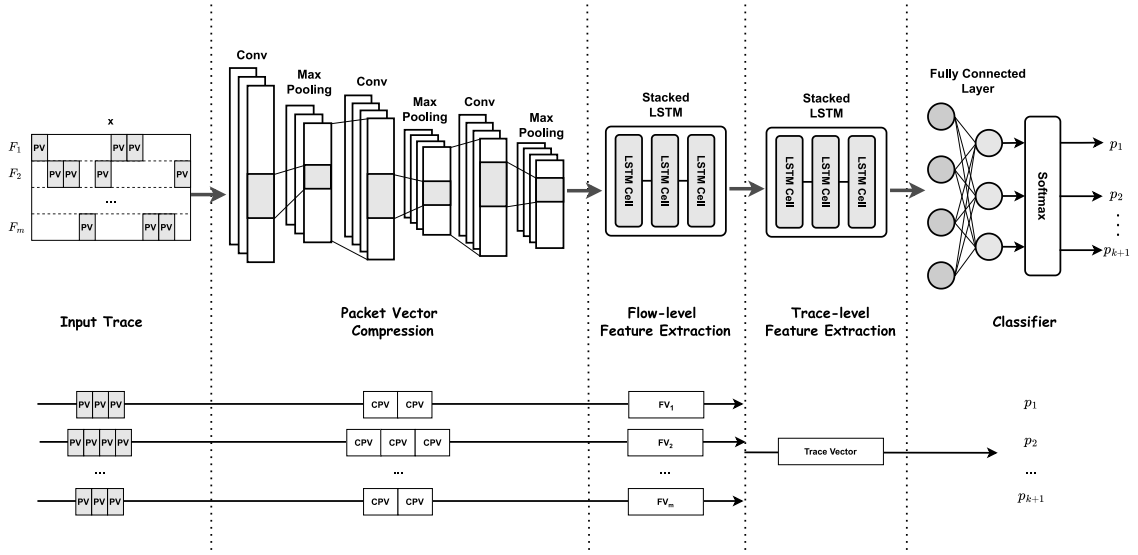


Fig. 3. Structure of discriminator network and transformation of input.

Feature extraction at flow and trace level via recurrent neural networks. Utilizing the Recurrent Neural Network (RNN), we are able to convert a sequence of variable length into a fixed-length vector by retaining only the final hidden state. We have several choices in terms of which type of RNN to employ. Traditional recurrent neural networks (RNNs) are capable of learning long-term dependencies in input sequences. However, they often encounter the vanishing gradient problem during backpropagation. While Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) networks address this issue to some extent, the problem is not entirely eliminated. Therefore, prior work proposed a bi-directional LSTM which significantly reduces the vanishing gradient problem by halving the average propagation path length. For this reason, we employ stacked bi-directional LSTM layers in the discriminator to capture flow- and trace-level features of each obfuscator.

As shown in Fig. 3, the first LSTM module extracts the flow-level features from each flow of packet vectors and converts the input into a set of fixed-size flow vectors (FVs). In the second LSTM module, a variable-length sequence of flow vectors within the same trace is mapped into a fixed-length trace vector, during which information at the trace layer is abstracted.

Class probabilities prediction with fully connected neural network. Given a trace vector produced by the LSTM-based feature extraction, the classifier comprised of fully-connected layers and a Softmax function predicts the probabilities of the trace belonging to each specific class. The class associated with the index of the highest value in the Softmax output is considered to be the predicted class.

In a classification task with K classes, the discriminator outputs a probability distribution over $K+1$ categories: the first K represent the probabilities of real classes, while the last indicates the probability of a sample being synthetic. During training, all outputs are used as they are. However, during validation and testing, the discriminator is treated as a standard classifier of obfuscated traffic, thus the $(K+1)^{th}$ output is simply ignored.

4.4. Generator

The generator can take two possible approaches to synthesizing useful training signals: (1) directly generating synthetic samples that structurally align with real traces, which, in our case, requires preserving trace-, flow-, and packet-level characteristics; and (2) learning from the feature space to generate feature vectors with similar statistical properties to real trace-derived representations. While the former enables precise reconstruction of real samples, which is an essential capability for tasks such as image synthesis, it increases model complexity, raises training costs, and may introduce instability. Given that our work aims to train an effective discriminator (classifier) for obfuscated traffic classification on imbalanced datasets, the generator synthesizes minority-class feature vectors at the feature-space level to strengthen the minority-class training signal, as illustrated in Fig. 2.

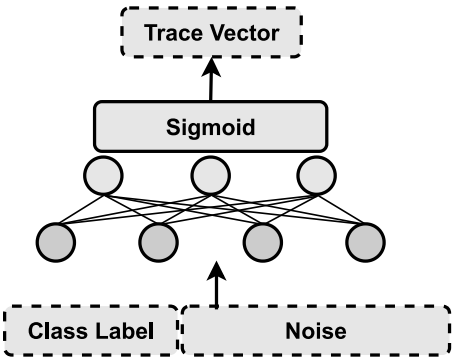


Fig. 4. Structure of class-conditioned generator network.

To synthesize samples in feature space, a simple Multilayer Perceptron (MLP) can be used, as shown in Fig. 4 (note that the number of layers and neurons depicted does not reflect the actual model hyperparameters). To guide the generator in producing samples of specific classes and mitigate mode collapse to some extent, we encode the target class label in a one-hot format within the generator's input noise and incorporate cross-entropy constraints in the loss function to optimize the generator's learning objective.

5. Experimental setup

5.1. Dataset collection and preprocessing

We set up an automated system for data collection as illustrated in Fig. 5. Python scripts were developed using Selenium to automate Google Chrome in sending requests to a curated list of the 10,000 most accessed websites. The popularity rankings provided by Tranco (Le Pochat et al., 2019) were used because they are designed to be more resistant to ranking manipulation than Alexa-style lists.

The browser, Tor client, and obfuscation PTs are encapsulated into a Docker image. By modifying the configuration file *torrc*, Tor is configured to route traffic through different PTs. The Linux command *nsenter* is used to execute *tcpdump* within the network namespace of the running Docker container, thereby capturing traffic entering and exiting the container while reducing background traffic. The use of Docker containers and Linux namespaces also enables multiple data collection instances to run in parallel.

The captured traffic was persisted to PCAP files. We preprocess the captured traffic to eliminate packets unrelated to the classification task and extract flows in accordance with the *trace format* outlined in Section 4. More specifically, we take the following steps to produce a dataset suitable for our model.

1. **Remove irrelevant packets.** Packets that do not belong to TCP or UDP transport-layer traffic are excluded. For the retained TCP and UDP packets, we extract metadata needed for packet vectorization, including packet size, direction, relative timestamp, and inter-arrival time. This rule preserves DNS query and response packets generated by Dnstt, which are UDP-based rather than DTLS-based, while also preserving UDP packets used by Snowflake's WebRTC/DTLS communication. While previous traffic fingerprinting studies often discard packets smaller than a specified size threshold, certain obfuscation protocols (e.g., Dnstt) do generate smaller packets. Discarding these would omit crucial features carried by obfuscated traffic. Therefore, no packet size threshold is applied in this work.
2. **Segment traffic into traces.** Traffic is segmented into traces as detailed in the section immediately following. Packets are also converted to Packet Vectors (PVs) during this phase.
3. **Discard short traces.** Traces with insufficient packet vectors (average flow length < 32) are deemed inadequate for feature extraction and are therefore discarded.
4. **Prepare dataset.** Traces are shuffled and subsequently divided into training, validation, and test sets. Each constructed trace is assigned to exactly one subset, thereby preventing direct trace overlap across the three subsets. The available collection metadata vary in session-level granularity across PTs; accordingly, the present evaluation adopts a consistent trace-level partitioning protocol under the controlled collection setting. A summary of the resulting dataset is provided in Table 1.

5.2. Trace segmentation

In the context of traffic analysis, a flow or connection is usually identified by the 5-tuple abstracted from the headers at *Transport* and *Network* layers:

⟨protocol, src IP, dest IP, src port, dest port⟩

which generally corresponds to a single communication behavior between a pair of entities. For TCP protocol flows, the process typically begins with the "three-way handshake" for connection establishment and ends with the "four-way handshake" for connection termination. In contrast, since the UDP protocol lacks an explicit connection maintenance process, flow boundaries are generally determined based on packet inter-arrival times.

However, the Tor client employs circuit multiplexing to reduce the overhead associated with frequently establishing new connections. With circuit multiplexing, communication data within the same connection window is routed through a single circuit to the exit node, causing the 5-tuples across different connections to be identical. This makes the data indistinguishable from an external observer's perspective.

To this end, two time-based trace segmentation methods are employed, namely SPAN and INTERVAL. Specifically, SPAN groups packets within fixed time spans into traces, while INTERVAL defines trace boundaries based on packet arrival time differences exceeding a set threshold. These two methods are evaluated through experiments in Section 5. In addition, a fixed time interval is intentionally introduced between individual requests during data collection to facilitate more precise trace segmentation within the collected dataset.

The purpose of trace segmentation is two-fold. First, it extracts sufficient samples from the raw traffic streams for model training, while ensuring that the size of each sample remains computationally manageable. Second, in real-world scenarios, it is unrealistic to assume that only a single type of obfuscator is active throughout an entire trace. Trace segmentation can help isolate activities of individual obfuscators.

While seemingly straightforward, selecting an appropriate time interval for partitioning flows and traces can be challenging. The chosen interval must strike a balance: it should be large enough to capture representative traffic features, yet small enough to minimize computational overhead as well as reduce interference between traffic generated by multiple obfuscators. For SPAN, we use a 5-second total-time threshold; for INTERVAL, we use a 0.5-second inter-arrival-time threshold.

Fig. 6 depicts the Cumulative Distribution Function (CDF) of trace lengths (i.e., the total number of packets in a trace) for datasets generated by the two trace segmentation methods. Formally, the *CDF* of a real-valued random variable X is given by

$$F_X(x) = P(X \leq x) \tag{10}$$

where the right-hand side represents the probability that the random variable X takes on a value less than or equal to x.

The distributions indicate that traffic generated by different obfuscators displays distinct patterns even at the packet level. For instance, DNS-tunneled traffic tends to include many small packets, while HTTPS-like proxy transports often show packet-size and timing patterns closer to ordinary encrypted web traffic.

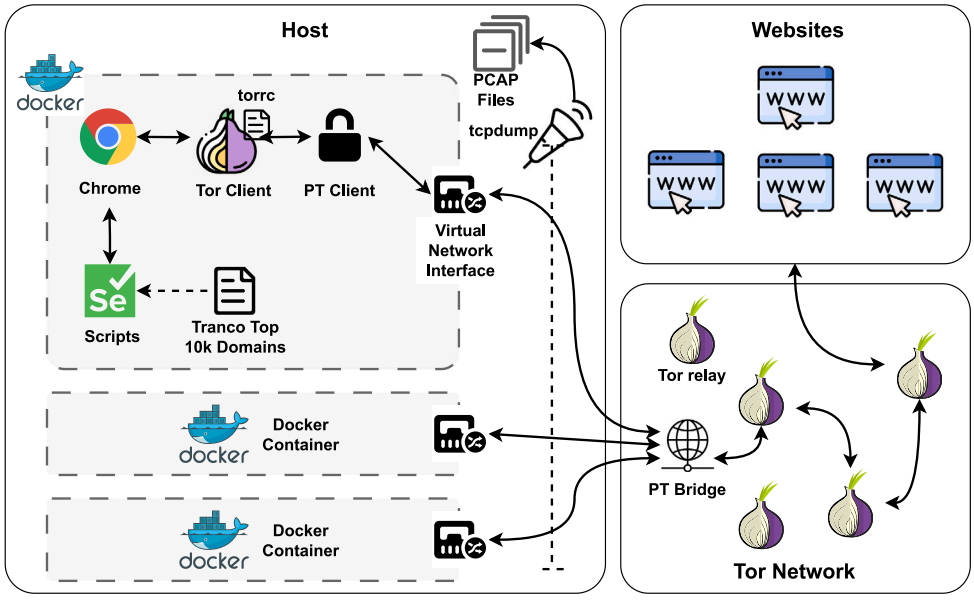
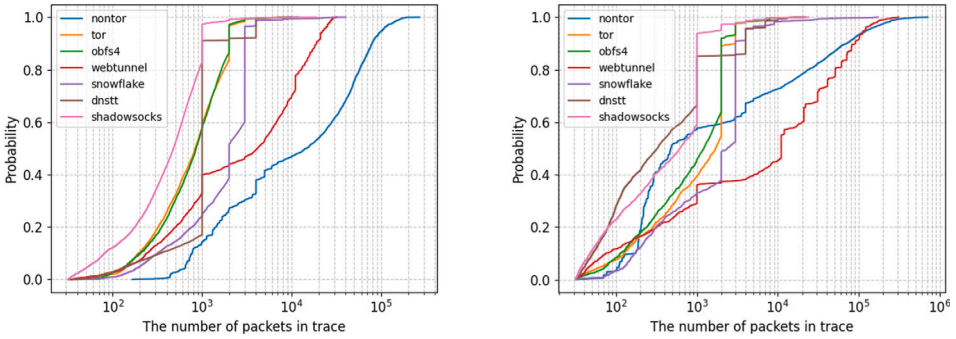


Fig. 5. Automated data collection system.

Table 1
Summary of the dataset.

PT type	Size (GB)	Number of traces	Training set (80%)	Validation set (10%)	Test set (10%)
Non-Tor	12.0	2010	1608	201	201
Tor	4.1	3544	2834	355	355
Obfs4	4.8	2911	2,329	291	291
WebTunnel	4.1	2173	1,737	218	218
Snowflake	5.9	4895	3,915	490	490
Dnstt	11.0	4846	3872	487	487
Shadowsocks	4.2	3406	2,724	341	341



(a) SPAN segmentation (b) INTERVAL segmentation

Fig. 6. CDFs of trace lengths under the two segmentation methods.

5.3. Model implementation

CNNs require the input sequence to have a sufficient length to yield meaningful results. In situations where a flow contains fewer packet vectors than needed, zero-padding is applied to extend the flow to the minimum required length. This padding standardizes the input shape and did not degrade performance in our experiments.

Training GANs presents several difficulties, primarily due to two key factors. First, instability can arise if the discriminator learns too quickly, leading to the generator's loss stagnating at high values. Second, the generator might either experience mode collapse, producing only a single type of output, or display erratic shifts between different data modes throughout training.

To improve stability and mitigate these problems, we adopt two key techniques. The first involves applying batch normalization (Ioffe and Szegedy, 2015) to all fully connected layers of both the generator and the discriminator, excluding the final output layers. This normalization helps regulate activation values at each layer and reduces the generator's tendency to switch unpredictably between modes while also enhancing the discriminator's generalization capacity. Considering the difficulties of applying batch normalization to recurrent neural networks, we also explored the possibility of incorporating layer normalization (Ba et al., 2016) into the LSTM modules of the discriminator network, but it did not improve performance in our setting, as shown in Section 6.4.

Table 2
F1-score point estimates of different obfuscated traffic identification methods on the near-balanced dataset.

Method	Non-Tor	Tor	Obfs4	WebTunnel	Snowflake	Dnstt	Shadowsocks	Macro-F1
Yao et al. (2020)	0.89	0.83	0.83	0.72	0.97	0.93	0.73	0.84
Xu and Zou (2021)	**0.95**	0.81	0.82	0.71	**0.98**	0.98	0.77	0.86
Xue et al. (2024)	**0.95**	**0.89**	**0.86**	0.79	0.97	**1.00**	0.80	**0.89**
Ours	**0.95**	0.88	**0.86**	**0.80**	0.95	0.99	**0.83**	**0.89**

5.4. Evaluation metrics

As obfuscated traffic identification is a multi-class classification task, we use the following metrics to evaluate our model's performance relative to others.

Precision is defined as the proportion of true positive predictions for a class to the total predictions made for that class.

$$Precision_i = \frac{TP_i}{TP_i + FP_i} \tag{11}$$

Recall (sensitivity) is defined as the proportion of true positive predictions for a class to the actual instances of that class.

$$Recall_i = \frac{TP_i}{TP_i + FN_i} \tag{12}$$

F1-score is the harmonic mean of precision and recall for a class.

$$F1 - Score_i = 2 \times \frac{Precision_i \times Recall_i}{Precision_i + Recall_i} \tag{13}$$

In addition, we employ the Macro variants of these metrics to reflect the general performance across all classes. Formally, Macro-X, where X represents Precision, Recall, or F1-Score, is defined as:

$$Macro - X = \frac{1}{K} \sum_{i=1}^{K} X_i \tag{14}$$

6. Results & evaluations

Tables 2–6 report point estimates on the corresponding fixed test sets. Within each comparison, the methods are evaluated using common data partitions. Boldface denotes the largest point estimate in each column and does not indicate statistical significance. Small numerical differences, including the 0.01 Macro-F1 difference between conventional data augmentation and the proposed model in Table 4, are therefore interpreted descriptively. Repeated-run variability is reported for the trace-segmentation experiment in Section 6.5, based on three training runs.

6.1. Performance on near-balanced datasets

To characterize the proposed model's performance on a near-balanced dataset, we compare its point estimates with those of the encrypted traffic classification models proposed in Yao et al. (2020), Xu and Zou (2021), Xue et al. (2024).

Since the models proposed in Yao et al. (2020), Xu and Zou (2021) do not support heterogeneous traffic samples as input, an additional feature extraction step is required during data preprocessing. CICFlowMeter (Habibi Lashkari et al., 2017) is used to extract 12 statistical features from structured traces obtained after trace segmentation.

The classification F1 point estimates for different methods on the same dataset are presented in Table 2. The proposed model obtains a Macro-F1 of 0.89, matching the highest point estimate in the table. These results indicate competitive near-balanced performance under the stated evaluation protocol.

6.2. Performance on imbalanced datasets

Imbalanced datasets have long posed a fundamental challenge to machine learning classifiers. To optimize the loss function, models often tend to ignore minority class samples, leading to a significant drop in their recall and reduced generalization capability. To address this issue, researchers have proposed various strategies, including adjusting class weights in the loss function, oversampling and undersampling, SMOTE, and domain-specific data augmentation techniques.

Random oversampling (ROS) and undersampling aim to balance class distributions by duplicating minority class samples or discarding majority class samples. However, studies (Chawla et al., 2002) suggest that oversampling provides limited benefits when the number of samples is small, while undersampling risks discarding valuable information and may even increase the likelihood of overfitting.

SMOTE (Synthetic Minority Oversampling Technique) (Chawla et al., 2002), generates synthetic samples by interpolating between minority class instances and their nearest neighbors. Experimental results indicate that SMOTE helps classifiers learn more generalized decision boundaries in the minority class feature space, effectively mitigating overfitting risks. However, in high-dimensional data, the distance metric used in SMOTE becomes less reliable, reducing the effectiveness of interpolation. Additionally, if the minority class contains noise or outliers, SMOTE-generated samples may amplify these issues.

Domain-specific data augmentation techniques are also widely used to address dataset imbalance. As noted in Qu et al. (2023), common augmentation methods for network traffic data include cropping, dropping, and timing perturbation. Cropping extracts random subsequences from the original traces, dropping randomly removes packets, and timing perturbation modifies packet arrival times within a controlled range. These techniques not only help re-balance datasets but also simulate variations in network conditions, enhancing the classifier's ability to generalize and reducing the risk of overfitting.

To create a controlled imbalanced dataset, we select samples labeled as *Tor* and discard 90% of them from the training set while leaving the validation and test sets unchanged. This setting isolates a representative single-minority-class case and allows different rebalancing and augmentation strategies to be compared under the same evaluation distribution.

Tables 3 and 4 present end-to-end comparisons of the imbalance-handling pipelines evaluated in this study. In the conventional pipelines, an MLP classifier is trained either without imbalance treatment (Baseline) or with weighted loss, oversampling and undersampling, SMOTE, or domain-specific data augmentation. The proposed GAN pipeline instead incorporates the hierarchical CNN/BiLSTM discriminator described in Section 4. All pipelines share the same traffic preprocessing and data partitions. Given the difference in classifier architecture, these results characterize the overall performance of the respective pipeline implementations. Section 6.3 complements this system-level comparison with a backbone-controlled ablation that retains the hierarchical CNN/BiLSTM feature extractor and compares configurations with and without the generator.

The tables present recall and F1 point estimates on the test set, with the last columns reporting Macro-Recall and Macro-F1, respectively.

The baseline MLP has a Tor-class recall of 0.63. Weighted loss, SMOTE, and data augmentation yield higher Tor-class recall point estimates, whereas the combined oversampling and undersampling

Table 3
End-to-end pipeline recall point estimates on the imbalanced dataset; classifier backbones are shown explicitly.

Method	Classifier backbone	Non-Tor	Tor	Obfs4	WebTunnel	Snowflake	Dnstt	Shadowsocks	Macro-Recall
Baseline	MLP	0.94	0.63	**0.91**	0.79	**0.99**	0.99	0.95	0.89
Weighted Loss	MLP	**0.98**	0.64	0.89	0.74	**0.99**	**1.00**	**0.97**	0.89
Oversampling & Undersampling	MLP	0.97	**0.92**	0.54	0.76	0.91	0.99	0.61	0.81
SMOTE	MLP	0.96	0.86	0.76	0.81	0.96	0.98	0.79	0.87
Data Augmentation	MLP	0.94	0.78	0.88	0.80	0.98	0.96	0.95	0.90
Ours	Hierarchical CNN/BiLSTM	0.97	**0.92**	0.88	**0.82**	0.98	0.98	0.89	**0.92**

Table 4
End-to-end pipeline F1-score point estimates on the imbalanced dataset; classifier backbones are shown explicitly.

Method	Classifier backbone	Non-Tor	Tor	Obfs4	WebTunnel	Snowflake	Dnstt	Shadowsocks	Macro-F1
Baseline	MLP	0.97	0.76	0.80	**0.87**	**0.99**	**1.00**	0.85	0.89
Weighted Loss	MLP	**0.98**	0.75	0.82	0.81	**0.99**	0.99	0.86	0.89
Oversampling & Undersampling	MLP	0.81	0.75	0.70	0.72	0.95	0.95	0.72	0.80
SMOTE	MLP	0.95	0.79	0.83	0.78	0.97	0.98	0.83	0.88
Data Augmentation	MLP	0.95	0.84	0.86	0.84	0.97	0.97	**0.87**	0.90
Ours	Hierarchical CNN/BiLSTM	0.93	**0.88**	**0.87**	0.85	0.98	0.99	0.83	**0.91**

Table 5
Recall point estimates for the backbone-controlled ablation.

Method	Non-Tor	Tor	Obfs4	WebTunnel	Snowflake	Dnstt	Shadowsocks	Macro-Recall
Solo Discriminator	**0.98**	0.69	0.88	0.76	**0.99**	**1.00**	**0.96**	0.89
Solo Discriminator with Weighted Loss	0.94	0.74	**0.90**	0.78	**0.99**	**1.00**	0.94	0.90
Ours	0.97	**0.92**	0.88	**0.82**	0.98	0.98	0.89	**0.92**

Table 6
F1-score point estimates for the backbone-controlled ablation.

Method	Non-Tor	Tor	Obfs4	WebTunnel	Snowflake	Dnstt	Shadowsocks	Macro-F1
Solo Discriminator	**0.98**	0.79	0.83	0.80	0.98	**0.99**	**0.86**	0.89
Solo Discriminator with Weighted Loss	0.97	0.82	0.84	**0.87**	**0.99**	**0.99**	0.85	0.90
Ours	0.93	**0.88**	**0.87**	0.85	0.98	**0.99**	0.83	**0.91**

pipeline increases Tor recall while reducing several other class-level values.

At the end-to-end pipeline level, the proposed model yields a Tor recall of 0.92, a Tor F1 of 0.88, a Macro-Recall of 0.92, and a Macro-F1 of 0.91. The observed Macro-F1 difference from conventional data augmentation is 0.01 (0.91 versus 0.90) and is interpreted descriptively.

6.3. Backbone-controlled ablation study

To assess the generator's contribution while retaining the hierarchical feature-extraction backbone, we conduct an ablation study on the imbalanced dataset. All configurations retain the discriminator's CNN/BiLSTM feature extractor. The ablated variants remove the generator and the synthetic class from the discriminator output, treating the discriminator as a standard obfuscated traffic classifier; one variant uses the unweighted classification loss, and the other uses a weighted loss. Tables 5 and 6 report the resulting test-set point estimates.

Within this backbone-controlled ablation, the Tor-class recall point estimate is 0.92 when the generator is included, compared with 0.69 for the solo discriminator and 0.74 for its weighted-loss variant. The corresponding Tor-class F1 values are 0.88, 0.79, and 0.82, respectively.

6.4. Enhancing performance through BN & LN

To evaluate the impact of batch normalization and layer normalization on convergence speed and model performance, we conduct a comparative experiment with four different model configurations: (1) No Normalization (Baseline): no normalization techniques are used; (2) Batch Normalization Only (BN-Only): batch normalization is applied only to the fully connected layers of the generator and discriminator, without layer normalization; (3) Layer Normalization Only (LN-Only): layer normalization is applied only after the discriminator's LSTM layer, without batch normalization; (4) Full Normalization (Full-Norm): both batch normalization (for the fully connected layers of the generator and discriminator) and layer normalization (for the discriminator's LSTM layer) are used.

All experimental configurations use the same learning rate, batch size, and other hyperparameters to control for variables. Given that batch normalization relies heavily on the minibatch size to estimate the overall training set statistics, this experiment uses a relatively large minibatch size of 64 to maximize the effectiveness of batch normalization.

Model convergence and post-convergence performance are assessed by the model's prediction accuracy on the validation set after each epoch. However, since validation accuracy fluctuates slightly over time, determining the exact convergence point can be challenging. Two intuitive methods are used to define convergence:

- Windowed Accuracy Range: The accuracy variance within a sliding window over time is computed. If the variance drops below a set threshold, it is considered that the model has converged.
- Accuracy Change Trend: The rate of change in accuracy compared to the previous epoch is calculated. If the change rate remains below a threshold over multiple epochs, the model is considered to have stabilized and converged.

Fig. 7 presents the final experimental results. The convergence time (marked by an asterisk) is determined using the first method, with a window size of 10 and a fluctuation threshold of 1%.

Fig. 7 suggests that batch normalization plays a key role in improving model performance. Compared with the configuration without batch normalization, it accelerates convergence in the early stages of training and increases validation accuracy by more than 10% after convergence. However, the effect of layer normalization is limited in this experiment, and some configurations with layer normalization

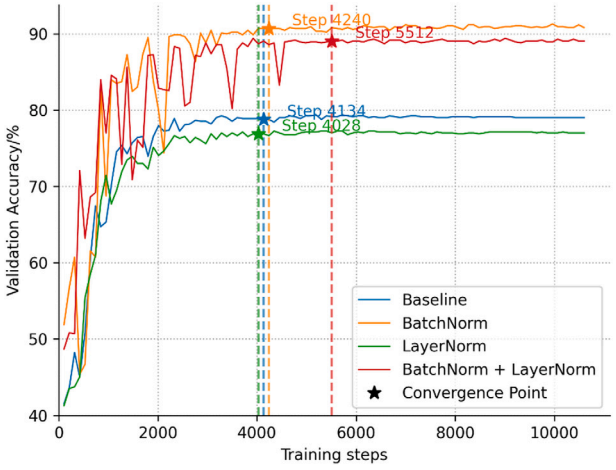


Fig. 7. Impact of BN & LN on performance.

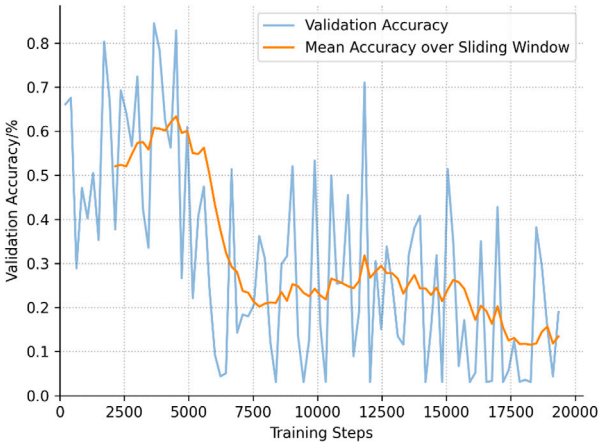


Fig. 8. Updating the running average using synthesized feature representations leads to model collapse.

show lower validation accuracy. In the other experiments throughout this paper, batch normalization is applied to the fully connected layers of both the generator and discriminator, while layer normalization is omitted.

During the experiments, we observed that when synthetic feature representations generated by the generator are fed into the discriminator, enabling batch normalization in the discriminator's fully connected layers (i.e., updating the mean and standard deviation estimates based on the input batch) leads to reproducible training collapse. The discriminator tends to classify all samples as the same category, resulting in a decrease in the model's accuracy on the validation set, as shown in Fig. 8. This specific configuration was repeatedly tested, and adjustments to the training hyperparameters did not resolve the issue.

The possible cause of this issue is that when batch normalization is enabled in the discriminator, its running mean and variance are simultaneously influenced by both real trace-derived representations and synthetic feature representations. If the generator has not yet converged, the synthetic representations may have unstable quality or a significant distribution mismatch with the real representations. This can bias the batch normalization statistics, leading to abnormal hidden-state distributions in the discriminator and ultimately resulting in model collapse (e.g., classifying all samples as the same class). In adversarial training, the quality of the generator directly affects the discriminator's learning. If the generated representations are of poor quality in the early stages, the discriminator may prematurely

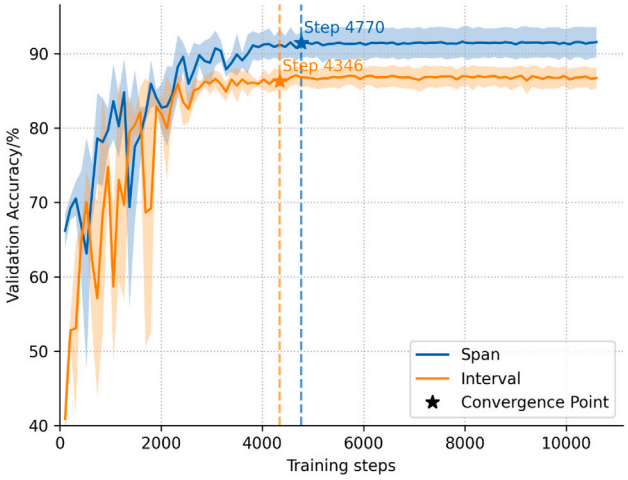


Fig. 9. Impact of trace segmentation methods on performance.

distinguish them from real representations (e.g., outputting extreme values close to 0 or 1), causing the batch normalization statistics to be dominated by low-quality synthetic representations and further exacerbating the discriminator's failure.

Therefore, in subsequent model training, we freeze the batch normalization statistics before synthetic representations are fed into the discriminator. Experiments show that this modification results in more stable training, and when the hyperparameters are appropriately set, the generator and discriminator reach a stable adversarial training state.

6.5. Impact of trace segmentation methods on performance

To evaluate the impact of trace segmentation methods on sample expressiveness and model performance, two methods, "Total Time Threshold Segmentation" (SPAN) and "Interval Time Threshold Segmentation" (INTERVAL), were applied to preprocess the same original traffic, resulting in two contrasting datasets. During the trace segmentation, the total time threshold was set to 5 s, while the interval time threshold was set to 0.5 s.

Following the selected model configuration from the previous section, batch normalization is applied to all hidden layers of the fully connected layers in both the generator and discriminator, while layer normalization is omitted. The model is then trained on the datasets generated using the two segmentation methods, with three separate training runs. The accuracy of the model on the validation set is recorded at each epoch, and the results are plotted as a line graph of accuracy over training steps, as shown in Fig. 9.

In the graph, the solid line represents the average of the three runs, while the semi-transparent shaded area indicates the standard deviation of the results across the runs. The model's convergence point is marked with an asterisk, with the determination of convergence following the criteria outlined in Section 6.4.

It can be observed that under the same conditions, the convergence speed of the two trace segmentation methods is nearly identical (with the convergence criteria being the same as in the previous section). After convergence, the model accuracy for SPAN is approximately 6% higher than that of INTERVAL, with stable and consistent results across three experiments. The semi-transparent shaded areas should be interpreted as variability indicators across repeated runs; our conclusion is therefore limited to the post-convergence trend in Fig. 9, where SPAN achieves higher validation accuracy than INTERVAL in these runs.

Additionally, a qualitative analysis of the two trace segmentation methods in an online environment shows that "Interval Time Threshold Segmentation" is more sensitive to packet delays caused by network

fluctuations, requiring careful parameter tuning to avoid insufficient or unusable trace lengths. In contrast, "Total Time Threshold Segmentation" is less sensitive to such delay variations in our setting. Moreover, as shown in Fig. 6, traces generated using SPAN are shorter, place lower demands on system computational power, and result in faster inference and quicker responses.

Accordingly, SPAN is adopted as the preferred trace segmentation method in our task setting.

7. Discussion

7.1. Ethical considerations and dual use

Obfuscated Tor traffic identification is a dual-use research topic. The same technique may support legitimate network measurement, enterprise security monitoring, and the evaluation of pluggable transports, but it may also be used by censors or other adversarial parties to improve traffic blocking. We therefore frame the contribution of this work as a measurement and classification study rather than a deployment-ready censorship system. The model uses observable traffic metadata such as packet size, direction, and timing; it does not decrypt payloads, inspect application content, identify individual users, or infer specific activities inside the Tor network. Consequently, it cannot determine whether a Tor user is accessing a particular hidden service or engaging in a specific type of online activity.

From a constructive perspective, understanding the distinguishability of current obfuscation tools can help researchers and transport designers identify weaknesses in deployed PTs and design stronger anti-censorship mechanisms. For network operators, the method may also support policy-compliant traffic analysis in managed environments. At the same time, we recognize the potential negative consequences of traffic-identification research. For this reason, raw packet captures are not publicly released at this stage, and any sharing of processed materials is subject to institutional review and privacy-preserving sanitization.

7.2. Evaluation scope

The experiments are designed to evaluate the proposed model under a multi-class obfuscated traffic classification setting, with particular emphasis on a controlled single-minority imbalance scenario. In this setting, Tor is treated as the minority class in the training set while the validation and test distributions are kept unchanged. The common partitions provide a consistent test basis for the end-to-end pipeline comparisons in Tables 3 and 4; the backbone-controlled ablation in Section 6.3 provides a focused assessment of the generator within the hierarchical feature-extraction backbone. Because Tables 2–6 report point estimates, small differences are interpreted descriptively. Repeated independent runs would provide a complementary characterization of stochastic variability in the principal classification comparisons. Broader imbalance patterns, such as multiple simultaneous minority PTs and extremely rare emerging transports, provide valuable directions for further evaluation.

During data collection, the browser, Tor client, and PTs are isolated in Docker containers and traffic is captured inside the corresponding network namespace, which reduces unrelated background traffic. The present experiments focus on class imbalance, while controlled background-traffic injection can further extend the evaluation. The trace-disjoint protocol prevents direct sample overlap and provides common evaluation partitions across all methods. Given the granularity of the collection metadata, traces acquired under related conditions may nevertheless retain residual correlation; the absolute performance values are therefore interpreted within the stated trace-level setting. A session- or collection-run-disjoint evaluation would complement the present analysis by assessing cross-session generalization.

8. Conclusion

This paper presents a semi-supervised GAN-based method for identifying obfuscated Tor traffic. The reported point estimates show competitive performance on the near-balanced dataset. In the controlled imbalance setting, the end-to-end pipeline comparison records a Tor recall of 0.92 and a Tor F1 of 0.88 for the proposed model. This end-to-end comparison characterizes the performance of the complete pipelines, including their respective classifier architectures. Within the backbone-controlled ablation, the generator-included configuration records higher point estimates for Tor recall and F1 than the two discriminator-only configurations. Future work will focus on expanding the evaluation to additional imbalance patterns, repeated independent runs, session- or collection-run-disjoint validation, and controlled background-traffic conditions.

CRediT authorship contribution statement

Tianbo Lu: Writing – review & editing, Supervision, Project administration, Methodology, Funding acquisition, Conceptualization. **Pengfei Zhao:** Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation. **Yingjie Cai:** Writing – review & editing, Validation, Investigation, Formal analysis. **Subatjan Abdulla:** Writing – review & editing, Validation, Methodology, Investigation. **Hanrui Chen:** Writing – review & editing, Validation, Methodology.

Funding

This work was supported by the National Key Research and Development Program of China [grant number 2023YFB3105902].

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

Al-E'mari, S., Sanjalawe, Y., Fraihat, S., 2023. Detection of obfuscated Tor traffic based on bidirectional generative adversarial networks and vision transform. Comput. Secur. 135, 103512. http://dx.doi.org/10.1016/j.cose.2023.103512.

Arjovsky, M., Bottou, L., 2017. Towards principled methods for training generative adversarial networks. arXiv:1701.04862.

Ba, J.L., Kiros, J.R., Hinton, G.E., 2016. Layer normalization. arXiv:1607.06450.

Bocovich, C., Breault, A., Fifield, D., Serene, Wang, X., 2024. Snowflake, a censorship circumvention system using temporary WebRTC proxies. In: 33rd USENIX Security Symposium (USENIX Security 24). pp. 2635–2652, URL: https://www.usenix.org/conference/usenixsecurity24/presentation/bocovich.

Chawla, N.V., Bowyer, K.W., Hall, L.O., Kegelmeyer, W.P., 2002. SMOTE: Synthetic minority over-sampling technique. J. Artificial Intelligence Res. 16, 321–357. http://dx.doi.org/10.1613/jair.953.

Frolov, S., Wustrow, E., 2020. HTTPT: A probe-resistant proxy. In: 10th USENIX Workshop on Free and Open Communications on the Internet (FOCI 20). USENIX Association, URL: https://www.usenix.org/conference/foci20/presentation/frolov.

Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., Bengio, Y., 2020. Generative adversarial networks. Commun. ACM 63 (11), 139–144. http://dx.doi.org/10.1145/3422622.

Habibi Lashkari, A., Draper Gil, G., Mamun, M.S.I., Ghorbani, A.A., 2017. Characterization of Tor traffic using time based features. In: Proceedings of the 3rd International Conference on Information Systems Security and Privacy - ICISSP, INSTICC. SciTePress, pp. 253–262. http://dx.doi.org/10.5220/0006105602530262.

Ioffe, S., Szegedy, C., 2015. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In: Proceedings of the 32nd International Conference on Machine Learning. In: Proceedings of Machine Learning Research, vol. 37, JMLR.org, pp. 448–456.

Le Pochat, V., Van Goethem, T., Tajalizadehkhoob, S., Korczyński, M., Joosen, W., 2019. Tranco: A research-oriented top sites ranking hardened against manipulation. In: Proceedings of the 26th Annual Network and Distributed System Security Symposium. In: NDSS 2019, http://dx.doi.org/10.14722/ndss.2019.23386.

Odena, A., Olah, C., Shlens, J., 2017. Conditional image synthesis with auxiliary classifier GANs. In: Proceedings of the 34th International Conference on Machine Learning. In: Proceedings of Machine Learning Research, vol. 70, PMLR, pp. 2642–2651.

Qu, J., Ma, X., Li, J., Luo, X., Xue, L., Zhang, J., Li, Z., Feng, L., Guan, X., 2023. An Input-Agnostic hierarchical deep learning framework for traffic fingerprinting. In: 32nd USENIX Security Symposium (USENIX Security 23). pp. 589–606.

Qu, J., Zhang, Y., Zhang, J., Li, J., Ma, X., 2026. Tracegram: Framing trace-level traffic analysis with temporally-aware multiple instance learning. In: 35th USENIX Security Symposium (USENIX Security 26). USENIX Association, URL: https://www.usenix.org/conference/usenixsecurity26/presentation/qu.

Radford, A., Metz, L., Chintala, S., 2015. Unsupervised representation learning with deep convolutional generative adversarial networks. arXiv:1511.06434.

Salimans, T., Goodfellow, I., Zaremba, W., Cheung, V., Radford, A., Chen, X., 2016. Improved techniques for training GANs. In: Proceedings of the 30th International Conference on Neural Information Processing Systems. NIPS '16, Curran Associates Inc., Red Hook, NY, USA, pp. 2234–2242.

Sheffey, S., Aderholdt, F., 2019. Improving meek with adversarial techniques. In: 9th USENIX Workshop on Free and Open Communications on the Internet (FOCI 19). USENIX Association, Santa Clara, CA, URL: https://www.usenix.org/conference/foci19/presentation/sheffey.

Springenberg, J.T., 2015. Unsupervised and semi-supervised learning with categorical generative adversarial networks. arXiv:1511.06390.

Wang, J., Cao, S., Zhao, J., Han, B., Shi, L., 2021. Identification of Tor anonymous network traffic based on machine learning. In: 2021 18th International Computer Conference on Wavelet Active Media Technology and Information Processing. ICCWAMTIP, IEEE, pp. 150–153. http://dx.doi.org/10.1109/ICCWAMTIP53232.2021.9674056.

Wang, L., Han, M., Li, X., Zhang, N., Cheng, H., 2021. Review of classification methods on unbalanced data sets. IEEE Access 9, 64606–64628. http://dx.doi.org/10.1109/ACCESS.2021.3074243.

Xu, W., Zou, F., 2021. Obfuscated Tor traffic identification based on sliding window. Secur. Commun. Netw. 2021 (1), 5587837. http://dx.doi.org/10.1155/2021/5587837.

Xue, D., Kallitsis, M., Houmansadr, A., Ensafi, R., 2024. Fingerprinting obfuscated proxy traffic with encapsulated TLS handshakes. In: 33rd USENIX Security Symposium (USENIX Security 24). USENIX Association, Philadelphia, PA, pp. 2689–2706, URL: https://www.usenix.org/conference/usenixsecurity24/presentation/xue-fingerprinting.

Yao, Z., Ge, J., Wu, Y., Lin, X., He, R., Ma, Y., 2020. Encrypted traffic classification based on Gaussian mixture models and Hidden Markov Models. J. Netw. Comput. Appl. 166, 102711. http://dx.doi.org/10.1016/j.jnca.2020.102711.

Zou, Z., Zhang, Y., Chen, J., Zhang, J., Wang, Z., Ju, L., Xu, R., 2023. Be like a Chameleon: Protect traffic privacy with mimicry. In: 2023 IEEE 22nd International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom). IEEE, pp. 1449–1454. http://dx.doi.org/10.1109/TrustCom60117.2023.00198.